

ReproBot: A Multi-Agent System for Automated Scientific Paper Replication

Second Progress Report

Aleyna Kütük
aleynakutuk10@gmail.com

Engincan Varan
engincanvaran@gmail.com

Gül Korkut
serifegulkorkut@gmail.com

Göktuğ Mert Özdoğan
goekmeroz@gmail.com

inzva AI Projects #10
22.08.2026

Abstract

The first progress report described ReproBot as a proposed four-agent pipeline for automated machine learning paper replication, supported by a literature review, an implementation-ready project plan, and an early Reader prototype. This report covers the implementation phase. Five of the seven planned components are now built and verified by execution: PDF extraction, a five-field structured Reader, a Coder that emits a self-contained training script, a Docker-sandboxed Runner, and an Orchestrator owning the shared-memory state and the Coder $\leftrightarrow$ Runner retry loop. Two papers – *Network In Network* and *Wide Residual Networks* – have been carried end to end, from PDF through to a training script executing inside a container and reporting metrics back. We report three substantive findings. First, the retry loop repairs real defects: a deliberately reintroduced runtime fault was triaged, fed back to the Coder as targeted feedback, and repaired on the first retry. Second, structured architecture extraction measurably outranks the language model’s pretrained priors – given *Network In Network*, whose contribution is a custom `mlpconv` layer with no memorized reference implementation, the Coder produced the correct layer cascade and global-average-pooling head rather than the generic convolutional stack its priors would favour. Third, several failure modes in LLM tool use are silent by construction and require deterministic guards rather than prompt engineering. We are explicit about what remains: the Critic does not exist, so no reproduced number has yet been compared against any paper’s claim, and the measured CPU throughput places a full-fidelity training run at roughly 22 days per paper, making GPU access a precondition for any fidelity result.

1 Introduction

The first progress report [1] established ReproBot’s motivation and design. The problem it addresses is unchanged: reproducing a machine learning paper’s empirical results is a mechanical task in principle and a labor-intensive one in practice, and the large majority of published work ships no usable implementation – across ICLR, ICML and NeurIPS 2024, only 16.7–21.2% of accepted papers included a working one [3]. The gap ReproBot targets is equally unchanged: no surveyed system combines parsing a paper’s claims, executing a faithful implementation, and explicitly verifying the reproduced numbers against the published ones with a repair loop driven by that comparison.

What has changed is that the system now exists in code. This report is therefore structured around evidence rather than intent. Section 3 describes the architecture as implemented, and where it departs from the plan. Section 4 documents each stage. Section 5 reports the measured results of running two papers through the full pipeline. Section 6 collects engineering findings that we believe generalize beyond this project. Section 8 states the limitations plainly, including the ones that prevent us from making any replication-fidelity claim at all. Sections 9 and 10 cover next steps and conclusions.

Since the first report, the project has produced 30 commits across five newly implemented pipeline stages totalling approximately 6,700 lines of Python, together with 21 logged delegations to specialized coding, research, review and validation subagents.

2 Positioning

The first report surveyed nine agentic-ML and paper-replication systems in detail. We do not repeat that survey; instead we record where this phase’s implementation choices followed those precedents, where they diverged, and what the divergences cost or

bought.

Following AutoReproduce on execution feedback. AutoReproduce’s ablation is the strongest available evidence that executing generated code matters more than inspecting it: removing the debug-and-refine loop moved an LLM-judged alignment score only slightly while nearly tripling the performance-gap metric [4]. We took this as decisive when sequencing the work, building the Runner and the execution-error loop before the Critic, and Section 5 reports a local instance of exactly the phenomenon that ablation predicts.

Diverging from AutoReproduce on edit granularity. That system applies line-range edits rather than regenerating a file, which reduces token cost and limits collateral damage. Our Coder regenerates the whole script on every retry. This was a deliberate simplification for a single-file target, and Section 8 records its cost: a successful repair can introduce a new defect elsewhere, and nothing compares the semantics of successive attempts.

Diverging from AutoP2C on repair scope. AutoP2C localizes a fault to a file before correcting it, machinery that exists because it targets a multi-file repository [5]. Our Coder emits one self-contained script, so localization has nothing to range over; the triage step supplies the diagnosis that localization would otherwise provide.

Retaining the tolerance gate as the differentiator. Neither system compares a reproduced number against the paper’s claim under an explicit numeric tolerance. That comparison remains ReproBot’s distinguishing feature, and it remains unimplemented (Section 8) — a fact we would rather state than obscure.

3 Implemented Architecture

The pipeline follows the planned Reader → Coder → Runner shape, with two departures worth stating.

The Reader emits five fields, not four. The project plan specified `method_summary`, `claims`, `hyperparameters` and `architecture_notes`. All four are implemented. A fifth, `data_pipeline`, was added on the plan’s own recommendation: data-processing detail is the single weakest-covered aspect of paper-to-code extraction across every surveyed system (56% coverage in PaperCoder against 79–92% elsewhere [3]), and is therefore treated as a first-class output field rather than folded into the method summary.

The Runner’s interface is a generated shell script, not a command line. The Coder emits a `reproduce.sh` alongside every training script, exposing four modes — `probe`, `smoke`, `capped` and `full` — and the Runner invokes nothing else. It never constructs a Python command and never passes a flag. This keeps the Runner paper-agnostic: a future

paper whose script requires entirely different arguments changes only its own `reproduce.sh`. The contract narrows from eight exactly-spelled command-line flags to four mode names.

The Orchestrator is implemented as a bounded, deterministic Python loop rather than the Lang-Graph state machine the plan proposed. At the present scope the graph would comprise three nodes and one conditional edge, which does not justify the dependency. The shared-memory object follows the plan’s schema exactly and the transition logic is isolated in pure functions, so migration remains mechanical once the Critic introduces genuine branching.

3.1 The shared-memory object

The Orchestrator owns one versioned object per paper, following the project plan’s schema. It is the artifact that lets the loop answer “have we already tried this” and that a future Report Generator will read without re-deriving anything.

Table 1: The shared-memory object as implemented. ✓ denotes a field populated today; ✗ a field present in the schema but not yet written by any stage.

Field		Contents
<code>paper_id</code>	✓	Stem of the source paper
<code>reader_output</code>	✓	All five extraction fields, embedded
<code>coder_output</code>	✓	Script path, version, diff from previous attempt
<code>runner_output</code>	✓	Status, reproduced metrics, log paths, error trace
<code>critic_output</code>	✗	Permanently null; reserved so the schema does not change when the Critic lands
<code>retry_count</code>	✓	Retries consumed
<code>retry_budget</code>	✓	Ceiling, default 2
<code>history</code>	✓	Append-only log of every stage transition, with timestamps
<code>attempts</code>	✓	Per-attempt record: version, feedback given, runner status, triage category, plateau ratio

Every attempt’s script is archived beside the state file, so a run’s full history is reconstructable after the fact rather than only summarized.

4 Stage Implementations

4.1 Extraction: ocr/

Four independent PDF-to-Markdown backends share one command-line shape. The vision-language backend renders each page to an image and sends

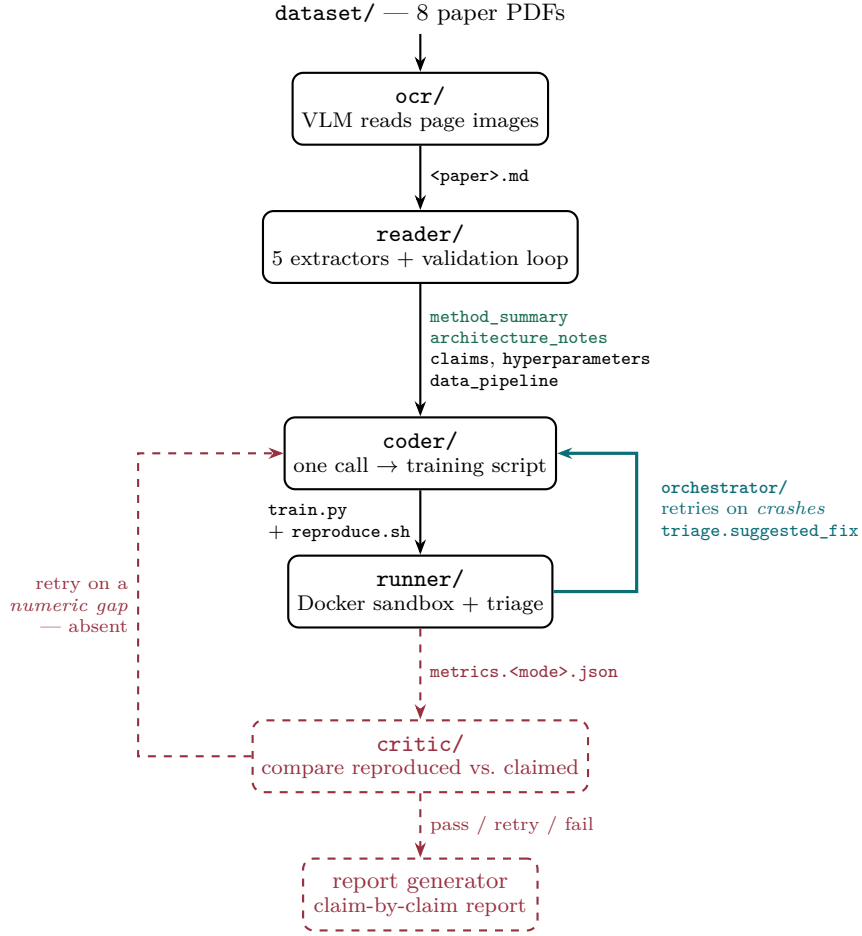


Figure 1: ReproBot as implemented. Solid components are built and verified by execution; dashed components in red do not exist. Two loops are shown and only one is real: the Orchestrator retries on execution failures, routing the Runner’s triage back to the Coder as feedback. The absent loop — retrying on a numeric gap between the reproduced value and the paper’s claim — is the one the literature review identifies as missing from every surveyed system, and it is missing here too.

it to Claude, so figures are genuinely read rather than inferred from surrounding text — a property the Reader’s architecture extraction depends on directly (Section 4.2). Six of the eight target papers are extracted. Two fail on a deterministic content-filter false positive, diagnosed as originating in the vision classifier rather than in our prompt: the same page fails identically at different render scales and under a minimal one-line prompt.

4.2 Structured extraction: reader/

Five extractors implement a shared abstract base class, so the pipeline treats every stage uniformly and a new extraction type requires one new class and one line in the stage list. A validator makes one further model call over the combined output and the paper text, flagging inconsistencies without fixing them; the pipeline routes each flag back to the single stage its `relates_to` prefix names, re-runs that stage with the flag folded into its prompt as feedback, and re-validates, capped at three passes.

4.2.1 The five extractors

Each extractor is a class implementing a shared abstract base: a name, and an `extract` method taking the paper’s Markdown, a client, and an optional feedback string. The uniformity is what makes the retry loop stage-agnostic — it calls `extract`, keys results by name, and on a validation flag routes it back to whichever stage owns that name. Adding `data_pipeline`, then `method_summary` and `architecture_notes`, each required one new class and one line in the stage list, with no change to the loop, the validator, or the routing.

Table 2: The Reader’s five extraction stages and their consumers.

Stage	Extracts	Consumed by
<code>method_summary</code>	Problem, core idea, novelty, prose summary	Coder content Report Generator
<code>architecture_notes</code>	Components, equations, scaling rule, unstated details	Coder (primary)
<code>claims</code>	Metric, dataset, value, unit, source, variant	Critic (future)
<code>hyperparameters</code>	Name, value as string, unit, source, variant	Coder
<code>data_pipeline</code>	Per-dataset preprocessing, augmentation, splits; reference URLs	Coder

Two schema decisions recur across stages and are worth stating once. Values that could be numbers

are kept as *strings* — a learning-rate schedule reading “0.1 initial, dropped by 0.2 at epochs 60, 120 and 160” must survive verbatim rather than collapsing to a scalar, and “not stated” must remain expressible. And every extracted item carries a `source` naming the table, figure block, or prose section it came from, with a transcribed page number, so a downstream consumer or a human can check it.

4.2.2 The validation retry loop

After all five stages run, a validator makes one further call over the paper text and the combined output. It is prompted to *flag*, not *fix*, two classes of problem: inconsistencies between stages, and content in the paper that looks like it should have been captured and was not. Each flag carries a `relates_to` field.

The loop then routes. A flag whose prefix matches exactly one stage name is sent back to that stage, which re-runs with the flag’s text folded into its prompt — a full re-extraction addressing a specific complaint, not a patch to individual fields. A flag that does not map to exactly one stage, such as a cross-check between two of them, is left in the output as report-only and never auto-retried, because attributing a genuine cross-stage disagreement to one side is not a solved problem.

A worked example from an early run against *Wide Residual Networks*: the first pass extracted 73 claims, and the validator caught that one of them labelled a 4.00% value as a CIFAR-100 result when it was the paper’s CIFAR-10 value from the same table row, duplicated under the wrong dataset label. The claims stage re-ran with that complaint and the misattributed entry was removed. This is the loop working as designed, and it is a defect that no schema constraint would have caught.

4.2.3 Recording what the paper does not say

The `architecture_notes` stage carries a required `unstated_details` list. Its rationale differs from the rest of the pipeline’s “do not guess” discipline in one respect that we think is worth drawing out. Elsewhere, recording “not stated” is simply honest. Here it is load-bearing, because the consumer is a code generator and code has to run. An extractor that invents a plausible channel count does not produce an obvious error — it produces a model that trains successfully and reports a number for the *wrong network*. The failure is silent and the artifact looks correct. Making the gap explicit moves the decision downstream, where the Coder must choose a value and disclose that it chose it.

For *Network In Network* the stage records twelve such gaps, quoting the paper’s own deferral — “the detailed settings of the parameters are provided in the supplementary materials” — rather than supplying the values from elsewhere.

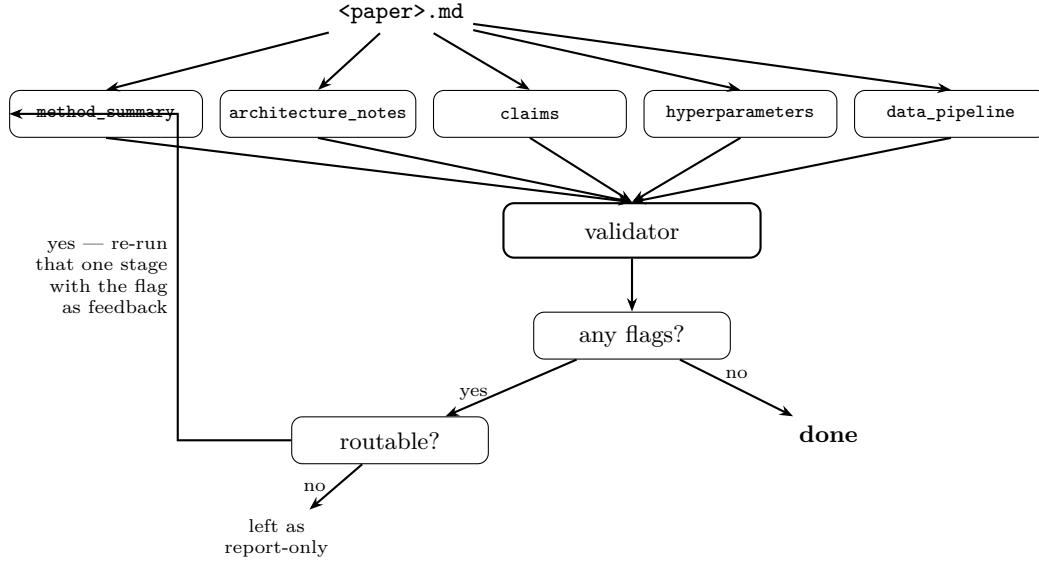


Figure 2: The Reader’s five extractors and its validation retry loop. All five read the same Markdown and none consumes another’s output, so they are independent and the order is only reading order. The validator sees the paper text plus all five results and flags rather than fixes. A flag whose **relates_to** prefix names exactly one stage is routed back to it; a flag spanning two stages is left as report-only, because attributing a genuine cross-stage disagreement to one side is not a solved problem. The loop is capped at three validation passes.

4.2.4 Distinguishing the paper’s own equations from its contrasts

A paper introducing a new layer typically prints the conventional formulation, and often a rival’s, beside its own. *Network In Network* prints all three within two pages: the conventional convolution it replaces, its own `mlpconv` computation, and maxout’s, which it argues against.

Returned as an undifferentiated list of LaTeX strings, these are three plausible specifications, and a code generator will implement one of them. Instructing the model to omit the baselines did not work — it continued to return them, which is unsurprising, since they are genuinely relevant context for understanding the method. Requiring the *schema* to carry an `is_own_method` flag on every entry did work immediately, and the classification proved correct across all four papers processed (Table 5).

The distinction survives into the generated code: the Coder implements only entries marked as the paper’s own, and Section 5 confirms that it did.

4.2.5 Shared guards for tool use

All six model calls in the Reader route through one module holding the request and its response guards, rather than each stage parsing responses itself. This consolidation was prompted by the failure modes in Section 6, and it means a fix lands once rather than six times. It also makes the failure reporting uniform: a guard names the stage that misbehaved, the keys it expected, and the keys it actually received.

4.3 Code generation: coder/

One model call converts a paper’s Reader output plus its Markdown into a self-contained HuggingFace **Trainer** script. The stage never executes what it writes and never imports a deep-learning framework: it calls the model and writes files, leaving execution entirely to the Runner. Three things determine what it produces — how its inputs are ranked, what is checked before a script is accepted, and the interface it emits alongside the script.

4.3.1 Input precedence

The Coder receives two documents: the paper’s Reader output and the paper’s Markdown. Its prompt ranks them explicitly, and adds a third rank that is not a document at all.

1. **architecture_notes**, **hyperparameters** and **data_pipeline** — validated, source-grounded, and authoritative wherever they carry a value.
2. The paper’s Markdown — corroboration, and the source for anything the extraction does not cover.
3. The model’s own recollection of a well-known architecture — last resort, and usable only when disclosed in the output’s **assumptions** field.

The third rank is unusual enough to justify. A code generator cannot decline to produce a value the way an extractor can: something must be written where a channel count goes. The ranking therefore does not forbid using prior knowledge, which would

be unenforceable, but requires that its use be visible. Section 5 reports what that looks like in practice.

4.3.2 Two deterministic gates

Before a generated script is accepted, two checks run. Neither involves a model call and neither costs anything.

The script must parse as valid Python; a failure writes the source to a separate path for inspection and marks the run failed, rather than emitting broken code. And every command-line flag the Runner depends on must literally appear in the script’s text, with the model’s own claim about which flags it defined cross-checked against that text, since a model can report a flag it did not write.

The limits of both gates are discussed in Section 6: parsing establishes that a script is syntactically well-formed, never that it runs.

4.3.3 The generated interface

Alongside every training script the Coder emits a `reproduce.sh` whose four modes are the Runner’s entire vocabulary (Table 3). The header of that file doubles as an audit trail, listing the targeted claim, every hyperparameter with the paper value behind it, and every assumption the Coder had to make.

Table 3: The four modes of the generated `reproduce.sh`. Each writes its own metrics file, so a cheap stage’s numbers cannot be mistaken for a real run’s.

Mode	Cost	What it establishes
probe	2 optimizer steps	Data $\rightarrow$ model $\rightarrow$ loss
smoke	1 epoch, small slice	Execution reaches evaluation and the metrics write
capped	5 epochs, 512 images	Training learns — accuracy climbs above chance
full	The paper’s own setup	The only numbers comparable to the claim; requires a GPU

Passing **full** supplies no flags at all, because every default in the generated script is already the paper’s own value. The cheaper modes are overrides.

4.4 Sandboxed execution: runner/

The Runner executes a generated script inside a container, escalating through the modes of Table 3 and stopping at the first failure. Three aspects proved less straightforward than expected — how the image is built and mounted, how a wall-clock budget is actually enforced on a container, and how a failure is classified — and are treated in turn below.

4.4.1 Container design

The image is built from a slim Python base with CPU-only PyTorch installed from the dedicated index, rather than from a vendor image carrying CUDA layers that a CIFAR-scale CPU run never

touches. The result is 1.69 GB and builds in approximately 224 s.

No generated code is copied into the image. The paper’s output directory is bind-mounted at run time, which keeps the image static across regenerations — the Coder can rewrite every script in the project without invalidating a layer — and is also how results return: the script writes its metrics file into the mounted directory, which is the host directory, so no copy step is required.

A shared dataset cache is mounted alongside. Its value is measurable: the same **probe** stage takes 1774 s against a cold cache and 166 s against a warm one, nearly all of the difference being a one-time 170 MB download that all eight target papers would otherwise repeat.

4.4.2 The timeout trap

Enforcing a wall-clock budget on a container is less direct than it appears, and the obvious implementation is wrong. Terminating the client process that launched the container does not stop the container: it is a child of the daemon, not of the launching process, and it continues running, holding its mounts and consuming CPU.

Every run is therefore launched with an explicit name chosen in advance — after the client dies, that name is the only remaining handle — and on expiry the container is killed by name, treating an already-reaped container as success. A second subtlety: the timeout exception carries whatever partial output it had collected as raw bytes even when the call was made in text mode, so decoding must be defensive or the handler reporting the timeout crashes while doing so. Budgets are per-stage and enforced independently, so a hung cheap stage cannot consume the budget of the expensive one behind it. They are calibrated against a measured run rather than estimated: an earlier estimate would have timed out on the first cold-cache execution, producing a failure indistinguishable from a hung script.

4.4.3 Triage

A failed run is classified by a single cheap model call into a defect in the generated script or a problem with the environment. Success and timeout are classified mechanically with no model call at all — the exit status already answers the question, and inviting a judgment where none is needed costs money on the common path and adds a way to be wrong.

The classification is not merely descriptive: it determines routing (Section 4.5.1). Logs are captured in full to disk and separately truncated for the model’s prompt, retaining both the head, where import and setup failures appear, and the tail, where the traceback is.

4.5 The retry loop: orchestrator/

The Orchestrator owns the shared-memory object and the loop. Its routing order is deliberate: an environmental error stops immediately rather than consuming retries, because regenerating a script cannot repair a broken container. A timeout likewise stops, since no triage is produced for timeouts by design and a slow but correct script must not be “repaired” into a different one. Only a recoverable error regenerates.

A plateau guard compares each regenerated script against its predecessor and stops early when they are near-identical, rather than exhausting the budget on a fix that is not changing anything — following PaperBench’s observation that agents plateau rather than making steady long-horizon progress [2].

4.5.1 Verdicts

The loop terminates in one of seven verdicts (Table 4). Three are success or benign stops; four are failures distinguished by what a retry could possibly achieve.

Table 4: Terminal verdicts. Only **recoverable** failures within budget produce another attempt.

Verdict	Meaning
success	A stage passed; nothing to retry
environment_error	The container or its inputs are at fault; regenerating the script would change nothing
timeout	Exceeded the wall-clock budget; no triage is produced, and a slow but correct script must not be rewritten
retry_budget_exhausted	Recoverable, but out of attempts
plateaued	The regenerated script was near-identical to the one that just failed
untriaged_error	Failed with no triage available to route on
coder_failed	Generation itself failed, e.g. the syntax gate

The check order matters and is fixed. An environmental error is tested *before* the retry budget, because consuming a retry on a broken container wastes an attempt on a fault the Coder cannot address. A timeout stops for a different reason: no triage exists to route on, and a script that is slow but correct must not be “repaired” into a different one.

4.5.2 The plateau guard

Before spending a container run on a regenerated script, the Orchestrator compares it line-wise against

its predecessor and stops early if they are near-identical, rather than exhausting the budget on a fix that is not changing anything. This follows PaperBench’s observation that agents plateau rather than making steady long-horizon progress [2].

One implementation detail proved to matter more than expected. The standard sequence matcher applies a heuristic that discards frequently repeated elements, which on source code discards common lines. Measured on the same pair of scripts, the heuristic reported a similarity of 0.9975 while the exact comparison reported 0.7625 — against a 0.98 threshold, the difference between stopping and continuing. The heuristic is disabled, and the measured ratio is logged on every retry so the threshold can be tuned from data rather than remaining a guess.

5 Results

5.1 Extraction coverage

Table 5 reports the Reader’s output across the four papers processed. The *own/eq* column is the most informative: it counts equations marked as the paper’s own against the total captured. *All Convolutional Net* prints six equations of which two are its own; *Wide Residual Networks* prints one and it is its own. A field that always returned every equation would be useless to a code generator.

Table 5: Reader output across four papers. *Comp.* is architecture components; *own/eq* is own-method equations over equations captured; *Gaps* is *unstated_details* entries; *Flags* is unresolved validation flags after the three-pass cap.

Paper	Claims	Hyp.	Data	Comp.	own/eq
Network In Network	12	17	4	7	1/3
All Convolutional Net	17	13	4	10	2/6
Deep Residual Learning	66	28	6	10	2/3
Wide Residual Networks	61	20	4	11	1/1

Validation does not converge. Every paper finishes with flags remaining after the cap, and the count is worst on the paper with the most claims. Inspection shows a substantial proportion are the validator raising a concern and dismissing it within its own description rather than identifying a real defect. We regard this as a genuine limitation of the current design rather than a tuning problem, and record it as such.

5.2 End-to-end execution

Both papers with generated scripts were carried through the Orchestrator. Both succeeded on the first attempt with no retries required (Table 6).

The roughly sixfold difference in wall clock reflects model size: WRN-28-10 carries approximately

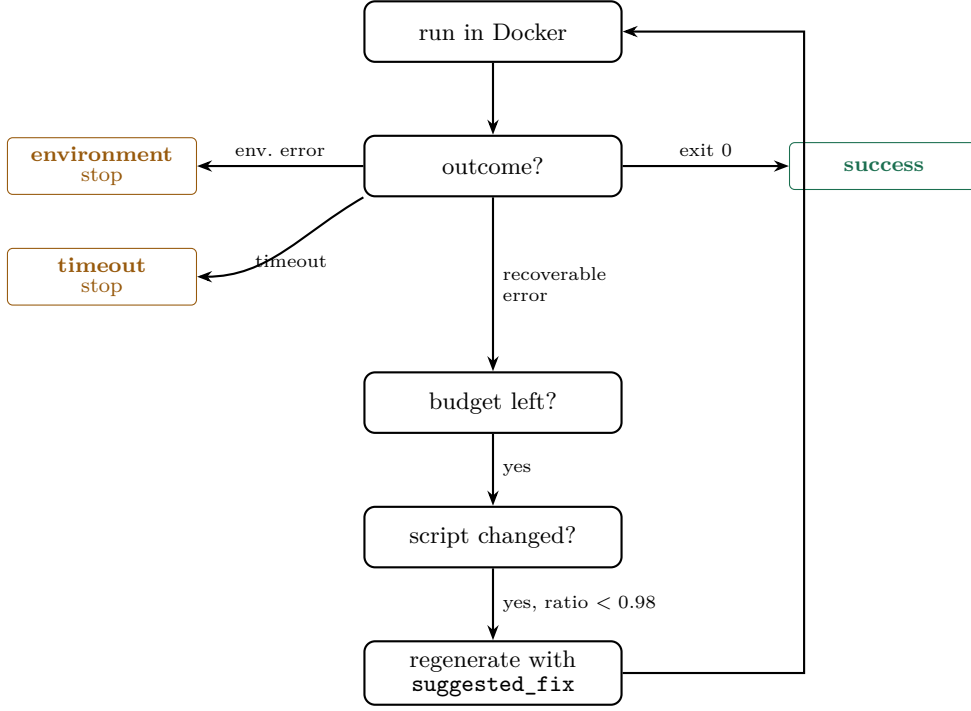


Figure 3: The Orchestrator’s routing. The order is deliberate: an environmental error stops *before* the retry budget is consulted, because regenerating a script cannot repair a broken container. A timeout also stops rather than retrying — no triage is produced for timeouts by design, and a slow but correct script must not be “repaired” into a different one. Only a recoverable error, within budget, whose regenerated script actually differs from its predecessor, results in another execution.

Table 6: End-to-end runs. Both papers reached **smoke** and returned **success** with zero retries consumed. Reproduced values are reported to demonstrate that the measurement path works end to end; they are *not* replication results (Section 8).

Paper	probe	smoke	Reproduced
Network In Network	32.2 s	62.1 s	86.72%
Wide Residual Networks	241.4 s	367.2 s	89.45%

36.5M parameters against *Network In Network*’s order of 1M. The reproduced error rates are far from the claimed values, and this is the expected and correct behaviour of the stages that produced them — **probe** and **smoke** train for one epoch over a few hundred images and exist to answer whether the script runs, never whether the number is right.

5.3 The retry loop repairs real defects

The loop was validated against a deliberately reintroduced fault: a learning-rate scheduler whose method reads an attribute its constructor never assigns. The fault is valid Python, so the syntax gate admits it, and it raises only when the training loop first logs the learning rate. This is precisely the class of defect the loop exists for.

The container run failed with an `AttributeError`; triage classified it as recoverable and produced a tar-

geted fix naming the class and the missing assignment; the `Coder` regenerated with that text folded into its prompt; the plateau guard measured a similarity of 0.41 against the previous attempt, well below the 0.98 stopping threshold; and the second attempt passed. The repair was not incidental — the regenerated script retained the class the triage named and added exactly the missing assignment. This is the local counterpart of AutoReproduce’s ablation, in which removing the debug loop barely moved an LLM-judged alignment score while nearly tripling the performance-gap metric [4]: code can read as a faithful implementation and still be wrong if it is never executed.

5.4 Architecture extraction outranks pretrained priors

The `Coder` was verified against *Network In Network* rather than *Wide Residual Networks*, deliberately. A wide residual network is among the most thoroughly memorized architectures in the literature and would be reproduced correctly from priors alone, demonstrating nothing about the extraction. *Network In Network* is the honest test: its `mlpconv` layer is the paper’s contribution, its dimensions appear nowhere in the paper, and its equation list contains two decoys.

The generated script contains a genuine `mlpconv` cascade — a $k \times k$ convolution followed by two 1×1

convolutions, with a docstring mapping each line to the paper’s Equation 2 — and contains no fully-connected layer at all, retaining the global-average-pooling classifier that is the paper’s second contribution. Both decoy equations are explicitly excluded and named as excluded.

The strongest evidence is negative. A priors-driven implementation of this paper is a plain convolutional stack with a fully-connected head; both of those defaults were avoided, and the generated script justifies the avoidance by citing components tagged `[baseline, not own method]` — a label from our extraction schema, not language from the paper.

We qualify this in one respect. The channel widths in the generated model are the canonical configuration, drawn from the model’s own knowledge rather than from the paper, which genuinely omits them. This is the designed outcome and it is disclosed in the output. The change made those values *visible* rather than *correct*, and a Critic will need to weight a disclosed prior below a paper-sourced value.

5.5 Cost

A Reader pass over one paper issues six model calls — five extractions and one validation — and each retry round adds one call per flagged stage plus one further validation. Observed runs use twelve to fourteen calls per paper against the three-pass cap. The Coder adds one call per generation attempt, and the Runner one cheap classification per genuine failure, which is zero on a clean run.

Two design choices hold this down. Triage is not invoked on success or timeout, so the common path costs nothing beyond generation. And the deterministic gates — syntax, required flags, plateau similarity — catch what they can without a model call at all.

The dominant cost is not the model but the compute, and it is not close: at the measured throughput, one full-fidelity training run exceeds the entire project’s model spend to date by orders of magnitude in wall-clock terms alone.

6 Engineering Findings

6.1 Silent failure modes in structured tool use

Three distinct failure modes were observed in which a model call returns a structurally valid response that is nonetheless empty or wrong, in a way indistinguishable from a correct extraction over a paper that had nothing to report.

The first is output truncation at the token limit, which yields a partial payload. The second is an intermittent well-formed response with every required field absent. The third is the most instructive: a *double-encoded* payload, in which the model returns a clean tool-use response whose single key holds the

entire result re-encoded as a JSON string. Field lookups then all miss and the extraction is discarded. It reproduced three times out of three on one paper, and fired again during later verification, where without the guard it would have reported zero datasets on an apparently clean run.

All three are now handled in one shared module rather than at six call sites. The general lesson is that structured-output failures do not necessarily announce themselves as errors, and a pipeline that treats a well-formed empty response as a legitimate negative result will silently accumulate them.

6.2 Schemas constrain models more reliably than instructions

Two independent cases: instructing the model to omit baseline equations did not stop it returning them, whereas requiring a per-entry `is_own_method` flag produced correct classification immediately; and a bookkeeping field the model never populated, despite being schema-required on every run, was eliminated by deriving it from data already present rather than by asking more insistently. Where a value is computable from what has already been extracted, computing it is more reliable than requesting it.

6.3 Static gates cannot establish that generated code runs

Two defects reached generated scripts that passed every deterministic gate. The first is the scheduler fault described above. The second, found by inspection, is a divergence between a script’s own bookkeeping — which describes one stage as $192 \rightarrow 192 \rightarrow 10$ — and the code it emitted, which builds $192 \rightarrow 10 \rightarrow 10$. The script runs, and the choice is a defensible reading of a dimension the paper never states; what is wrong is that the record describes a different network from the code.

Neither is reachable by parsing. Both argue for the execution-driven loop rather than for additional static analysis.

6.4 Platform constraints shaped the architecture

The development machine is an Intel Mac, and no PyTorch release supports both Python 3.13 and Intel macOS: 3.13 support arrived in the same release range that dropped Intel-macOS wheels. Reducing to Python 3.11 installs PyTorch 2.2.2, but current `transformers` requires 2.5 or newer and, on finding an older version, silently disables its PyTorch backend rather than raising.

This is why the Runner executes inside a container rather than on the host. The container runs its own Linux Python, so the constraint does not apply within it, and the project’s dependency lock stays free of PyTorch entirely.

7 Development Methodology

This phase was built through a delegation pattern that we record because it shaped the result, and because two of its failures are instructive.

Work was decomposed into scoped tasks dispatched to specialized subagents — research agents that survey precedent and propose a design without writing code, coding agents that implement a scoped change and verify it by running it, review agents that critique an artifact before a costly or hard-to-repeat action, and validation agents that audit existing output read-only. Each delegation, its brief, and its outcome is logged; 21 are recorded for this phase.

Two failures are worth reporting honestly, because both were caught only by independent verification rather than by the reports themselves.

In one case, a coding agent’s report asserted it had left four modules untouched; the version control diff showed all four had been refactored. The refactor was an improvement — it consolidated guards that had been duplicated — but the discrepancy meant the agent’s verification claims could not be relied upon, and the affected pipeline was re-run directly. In another, an agent predicted a specific failure by reading a “known issues” section and mistaking a historical, already-discarded defect for current state; the claim was false and a single search of the source disproved it.

The generalizable lesson is narrow but practical: an agent’s description of its own change is a claim about the world, and the cost of checking it against the repository is close to zero. We now treat the diff, not the report, as the record of what happened.

8 Limitations

We state these directly, because several bear on how the results above should be read.

No fidelity claim is possible yet. The Critic does not exist. Nothing in the system compares a reproduced number against a paper’s claim, and the field reserved for that verdict is permanently null. The reproduced error rates in Table 6 demonstrate that the measurement path works; they are not replication results and should not be read as any.

Full-fidelity runs are not currently feasible. Measured throughput is 5.256 training samples per second for WRN-28-10 on this CPU. The paper’s setup is 50,000 samples over 200 epochs, placing one paper and one claim at approximately 22 days. Every result reported here comes from stages that train for one epoch over a few hundred images. GPU access is a precondition for any fidelity measurement, not an optimization.

The evidence base is two papers. Both succeeded on the first attempt, so the retry loop’s behaviour in production remains demonstrated only by the injected fault. Four of the loop’s seven terminal

verdicts have been exercised only against test doubles.

Retries regenerate rather than patch. A repair rewrites the whole file, so a successful fix can introduce a new defect elsewhere, and nothing compares the semantics of successive attempts. AutoReproduce’s line-range edit mechanism [4] is the natural remedy and is not yet implemented.

Validation does not converge, as reported in Section 5, and two of the eight target papers cannot currently be extracted at all.

9 Next Steps

The Critic is the highest-value remaining component and closes the loop the project’s thesis rests on. Its verdict logic should remain simple, auditable arithmetic against an explicit numeric tolerance rather than a model judging numbers by eye — Agent Laboratory’s automated reviewer overestimated quality by 2.3 points against human judges [6], and an objective numeric check is precisely the failure mode ReproBot exists to avoid. It can be built and tested against synthetic metrics immediately; its verdicts become meaningful when compute allows a real run.

GPU compute should be secured in parallel rather than afterwards, since it gates every fidelity result.

Coverage should be extended across the remaining papers, which requires no new code and would convert single demonstrations into distributions.

The Report Generator is the promised deliverable and the cheapest remaining component: the shared-memory object already carries everything it needs, including the append-only history log.

10 Conclusion

ReproBot has moved from design to a working pipeline. Five of seven components are implemented and verified by execution; two papers have been carried from PDF to a training script running in a sandbox; the retry loop repairs real defects; and structured architecture extraction demonstrably outranks the language model’s priors on a paper where those priors would otherwise dominate.

What has been demonstrated is that the machinery works end to end. What has not been demonstrated — and cannot be, until the Critic exists and compute allows a full training run — is replication fidelity, which is the project’s actual claim. We regard stating that distinction clearly as more useful than reporting an aggregate that would obscure it.

References

- [1] Kütük, A., Varan, E., Korkut, G., & Özdoğan, G. M. (2026). ReproBot: A Multi-Agent System for Automated Scientific Paper Replication — First Progress Report. *inzva AI Projects #10*.
- [2] Starace, G., Jaffe, O., Sherburn, D., Aung, J., et al. (2025). PaperBench: Evaluating AI’s Ability to Replicate AI Research. *arXiv:2504.01848*.
- [3] Seo, M., Baek, J., Lee, S., & Hwang, S. J. (2025). Paper2Code: Automating Code Generation from Scientific Papers in Machine Learning. *arXiv:2504.17192*.
- [4] Zhao, X., Sun, M., Wang, W., et al. (2025). AutoReproduce: Automatic AI Experiment Reproduction with Paper Lineage. *arXiv:2505.20662*.
- [5] Lin, Z., Li, M., Chen, S., et al. (2025). AutoP2C: An LLM-Based Agent Framework for Code Repository Generation from Multimodal Content in Academic Papers. *arXiv:2504.20115*.
- [6] Schmidgall, S., Su, Y., Wang, Z., et al. (2025). Agent Laboratory: Using LLM Agents as Research Assistants. *arXiv:2501.04227*.
- [7] Li, R., Patel, T., Wang, Q., & Du, X. (2024). MLR-Copilot: Autonomous Machine Learning Research based on Large Language Models Agents. *arXiv:2408.14033*.